

本节内容

浮点数

加减运算

(溢出问题)

示例1：浮点数的加减运算“上溢”

```
int main() {  
    // 定义并初始化变量  
    float x = 170141183460469231731687303715884105728.0f; // 2127  
    float y = 255211775190703847597530955573826158592.0f; // 2127 + 2126  
    float z = x + y;  
  
    // 打印十进制值  
    printf("十进制值:\n");  
    printf("x = %.1f\n", x);  
    printf("y = %.1f\n", y);  
    printf("z = x + y = %.1f\n\n", z);  
}
```

z 溢出为 $+\infty$

$x=2^{127}$

$y=2^{127} + 2^{126}$

特殊状态的浮点数
(阶码全0 或 全1)

值的类型	单精度 float (32位)			
	符号	阶码	尾数	值
正零	0	全0	全0	+0
负零	1	全0	全0	-0
非规格化正数	0	全0	$f \neq 0$	$2^{-126}(0.f)$
非规格化负数	1	全0	$f \neq 0$	$-2^{-126}(0.f)$
正无穷大	0	全1	全0	$+\infty$
负无穷大	1	全1	全0	$-\infty$
无定义数 (非数)	0或1	全1	$f \neq 0$	NaN

十进制值：

```
x = 170141183460469231731687303715884105728.0  
y = 255211775190703847597530955573826158592.0  
z = x + y = inf
```

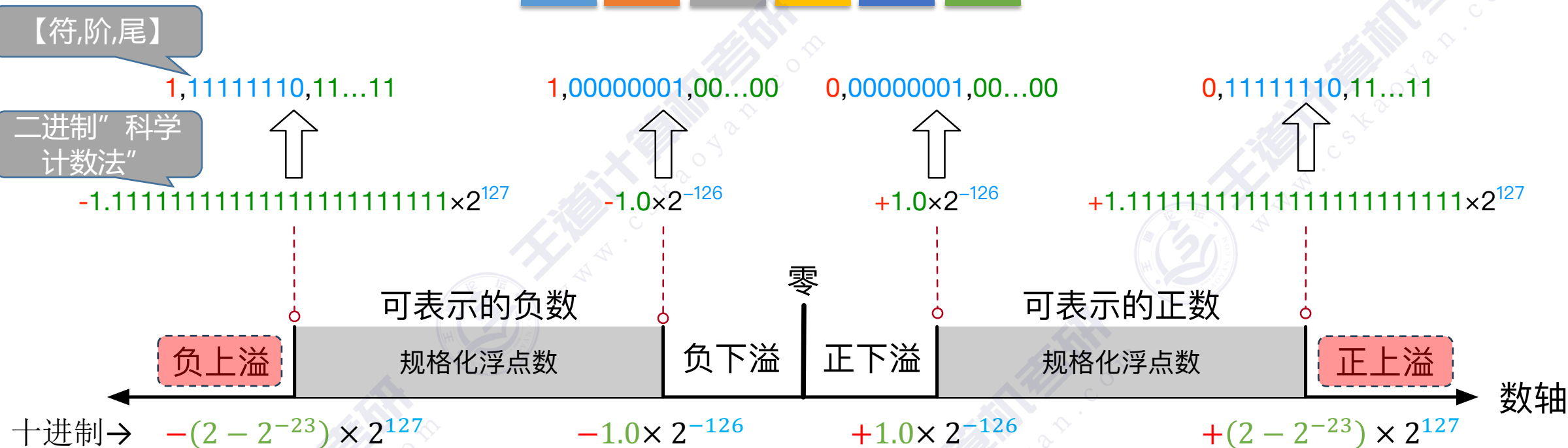
32位二进制表示:

x: 0 11111110 000000000000000000000000

y: 0 11111110 100000000000000000000000

z: 0 11111111 000000000000000000000000

IEEE 754: 浮点数的上溢 (Overflow)



运算结果大于最大规格化正数时称为正上溢，小于绝对值最大的规格化负数时称为负上溢。

正上溢、负上溢 统称上溢 (Overflow)，也会翻译为“溢出”

浮点数上溢 (溢出) 的处理:

① 浮点数运算部件将运算结果设置为 $+\infty$ 或 $-\infty$

② 设置浮点数溢出异常标志位 (例如: x86 会将浮点运算单元 FPU 的 OE 标志位 Overflow Exception 置为 1)

注: IEEE 754 规定, 默认不响应浮点数溢出异常, 不中断程序; 除非程序员手动开启此类异常响应

例1：浮点数加减运算“上溢”

已知 float $X = 2^{127}$, $Y = 2^{127} + 2^{126}$ 符合 IEEE 754 标准。分析计算机实现 float $Z = X + Y$ 的过程。

$X \rightarrow$ 0 11111110 000000000000000000000000

$Y \rightarrow$ 0 11111110 100000000000000000000000

尾数高位隐含1

+1.000000000000000000000000

← X 尾数原码

+1.100000000000000000000000

← Y 尾数原码

① 对阶 (原则：小阶向大阶靠齐) $X_{\text{阶}} - Y_{\text{阶}} = 0$, 说明X、Y阶码相等, 无需对阶

X尾数→

+1.000000000000000000000000

Y尾数→

+1.100000000000000000000000

② 尾数加减 (原码的加减运算)

原码加法, 得 Z 尾数原码→

+10.100000000000000000000000

③ 尾数规格化

尾数右规 (右移1位)

$Z_{\text{阶}} = \text{大阶} + 1$
 $= 11111110 + 1$
 $= 11111111$

Z 尾数原码→

+1.010000000000000000000000

④ 尾数的舍入处理

阶码全1, 说明发生溢出。运算结果记为“无穷大”

一旦指数上溢, 尾数就强制保存为全0

⑤ 溢出判断→ 阶码全为1说明发生上溢

【符, 阶, 尾】 = 【0 11111111 000000000000000000000000】

尾数保存为全0

$Z = +\infty$

例1：浮点数加减运算“上溢”

结论：浮点数的溢出不以尾数溢出来判断，尾数溢出可以通过右规操作得到纠正。运算结果是否溢出主要看运算结果的指数是否发生了上溢，因此是由指数上溢来判断的（右规后指数全1说明上溢）

已知 float $X = 2^{127}$, $Y = 2^{127} + 2^{126}$ 符合 IEEE 754 标准。分析计算机实现 float $Z = X + Y$ 的过程。

① 对阶（原则：小阶向大阶靠齐） $X_{\text{阶}} - Y_{\text{阶}} = 0$ ，说明X、Y阶码相等，无需对阶

X尾数→

+1.000000000000000000000000

Y尾数→

+1.100000000000000000000000

② 尾数加减（原码的加减运算）

原码加法，得Z尾数原码→

+10.100000000000000000000000

③ 尾数规格化

$Z_{\text{阶}} = \text{大阶} + 1$
 $= 11111110 + 1$
 $= 11111111$

尾数右规（右移1位）

Z尾数原码→

+1.000000000000000000000000

④ 尾数的舍入处理

阶码全1，说明发生溢出。运算结果记为“无穷大”

一旦指数上溢，尾数就强制保存为全0

⑤ 溢出判断→ 阶码全为1说明发生上溢

【符, 阶, 尾】 = 【0 11111111 000000000000000000000000】

尾数保存为全0

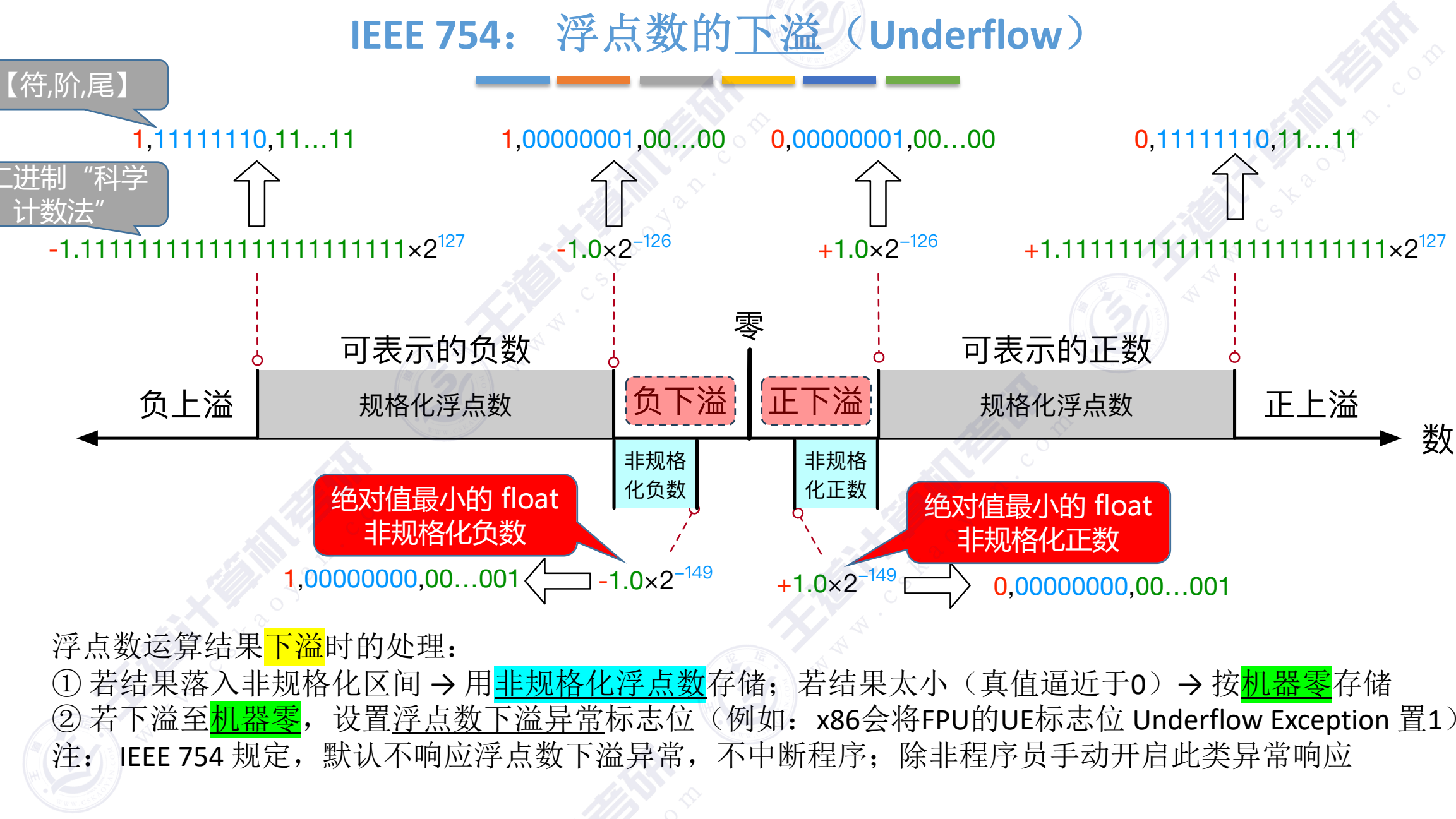
$Z = +\infty$

打印十进制小数点后60位，
就能看出效果

[illegible]

32位二进制表示:

z 下溢到非规格化浮点数区间

[illegible]

① 若结果落入非规格化区间 → 用 **非规格化浮点数** 存储；若结果太小（真值逼近于0）→ 按 **机器零** 存储
② 若下溢至 **机器零**，设置 **浮点数下溢异常标志位**（例如：x86会将FPU的UE标志位 Underflow Exception 置1）
注：IEEE 754 规定，默认不响应浮点数下溢异常，不中断程序；除非程序员手动开启此类异常响应

例2：浮点数加减运算“下溢”至非规格化区间

已知 float $X = 2^{-126} + 2^{-149}$, $Y = -2^{-126}$ 符合 IEEE 754 标准。分析计算机实现 float $Z = X + Y$ 的过程。

$X \rightarrow 0 \text{ } \underline{00000001} \text{ } 000000000000000000000000$

$Y \rightarrow 0 \text{ } \underline{00000001} \text{ } 000000000000000000000000$

尾数高位
隐含1

$+1.000000000000000000000001$

$\leftarrow X$ 尾数原码

$-1.000000000000000000000000$

$\leftarrow Y$ 尾数原码

① 对阶 (原则：小阶向大阶靠齐) $X_{\text{阶}} - Y_{\text{阶}} = 0$, 说明X、Y阶码相等，无需对阶

② 尾数加减 (原码的加减运算)

原码加法，得 Z 尾数原码 $\rightarrow$

$+0.000000000000000000000001$

③ 尾数规格化

如果尾数继续左移1位，阶码就要减1， $\underline{00000001} - 1 = 00000000$ 。而阶码全0表示“非规格化浮点数”，因此不能按照普通的“左规”方式处理。

$Z_{\text{阶}} = 00000000$

Z 尾数原码 $\rightarrow$

$+0.000000000000000000000001$

④ 尾数的舍入处理

如果当前阶码已经为 00000001 ，但尾数仍需左规，就直接将阶码置为全0。并且尾数不移位，保持原样

没有多余尾数，无需舍入

$+0.000000000000000000000001$

⑤ 溢出判断 $\rightarrow$ 阶码全0，说明发生下溢

【符, 阶, 尾】 = 【0, 00000000 , 000000000000000000000001 】

$Z = + 2^{-126} (0.f) = + 2^{-149}$